



Server Features

Resources

📄 Copy page



⚠️ You are viewing an older version (2025-06-18) of the specification. [View the latest version \(2026-07-28\)](#)

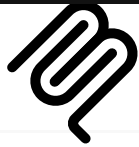
The Model Context Protocol (MCP) provides a standardized way for servers to expose resources to clients. Resources allow servers to share data that provides context to language models, such as files, database schemas, or application-specific information. Each resource is uniquely identified by a URI.

1. User Interaction Model

Resources in MCP are designed to be **application-driven**, with host applications determining how to incorporate context based on their needs.

For example, applications could:

- Expose resources through UI elements for explicit selection, in a tree or list view
- Allow the user to search through and filter available resources
- Implement automatic context inclusion, based on heuristics or the AI model's selection



Model Context Protocol



However, implementations are free to expose resources through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model.

2. Capabilities

Servers that support resources **MUST** declare the `resources` capability:

```
{
  "capabilities": {
    "resources": {
      "subscribe": true,
      "listChanged": true
    }
  }
}
```

The capability supports two optional features:

- `subscribe` : whether the client can subscribe to be notified of changes to individual resources.
- `listChanged` : whether the server will emit notifications when the list of available resources changes.

Both `subscribe` and `listChanged` are optional—servers can support neither, either, or both:



Model Context Protocol

```
}
```

```
{  
  "capabilities": {  
    "resources": {  
      "subscribe": true // Only subscriptions supported  
    }  
  }  
}
```

```
{  
  "capabilities": {  
    "resources": {  
      "listChanged": true // Only list change notifications supported  
    }  
  }  
}
```

3. Protocol Messages

3.1 Listing Resources

To discover available resources, clients send a `resources/list` request. This operation supports [pagination](#).

Request:



Model Context Protocol

```
"params": {  
  "cursor": "optional-cursor-value"  
}
```

Response:

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "result": {  
    "resources": [  
      {  
        "uri": "file:///project/src/main.rs",  
        "name": "main.rs",  
        "title": "Rust Software Application Main File",  
        "description": "Primary application entry point",  
        "mimeType": "text/x-rust"  
      }  
    ],  
    "nextCursor": "next-page-cursor"  
  }  
}
```

3.2 Reading Resources

To retrieve resource contents, clients send a `resources/read` request:

Request:



Model Context Protocol

```
"params": {  
  "uri": "file:///project/src/main.rs"  
}
```

Response:

```
{  
  "jsonrpc": "2.0",  
  "id": 2,  
  "result": {  
    "contents": [  
      {  
        "uri": "file:///project/src/main.rs",  
        "mimeType": "text/x-rust",  
        "text": "fn main() {\n    println!(\"Hello world!\");\n}"  
      }  
    ]  
  }  
}
```

3.3 Resource Templates

Resource templates allow servers to expose parameterized resources using [URI templates](#). Arguments may be auto-completed through [the completion API](#). This operation supports [pagination](#).

Request:



Model Context Protocol

```
"params": {  
  "cursor": "optional-cursor-value"  
}
```

Response:

```
{  
  "jsonrpc": "2.0",  
  "id": 3,  
  "result": {  
    "resourceTemplates": [  
      {  
        "uriTemplate": "file:///path",  
        "name": "Project Files",  
        "title": "📁 Project Files",  
        "description": "Access files in the project directory",  
        "mimeType": "application/octet-stream"  
      }  
    ],  
    "nextCursor": "next-page-cursor"  
  }  
}
```

3.4 List Changed Notification

When the list of available resources changes, servers that declared the `listChanged` capability **SHOULD** send a notification:

```
{  
  "jsonrpc": "2.0",  
  "method": "notifications/resources/list_changed"  
}
```



Model Context Protocol

Subscribe Request:

```
{  
  "jsonrpc": "2.0",  
  "id": 4,  
  "method": "resources/subscribe",  
  "params": {  
    "uri": "file:///project/src/main.rs"  
  }  
}
```

Update Notification:

```
{  
  "jsonrpc": "2.0",  
  "method": "notifications/resources/updated",  
  "params": {  
    "uri": "file:///project/src/main.rs"  
  }  
}
```

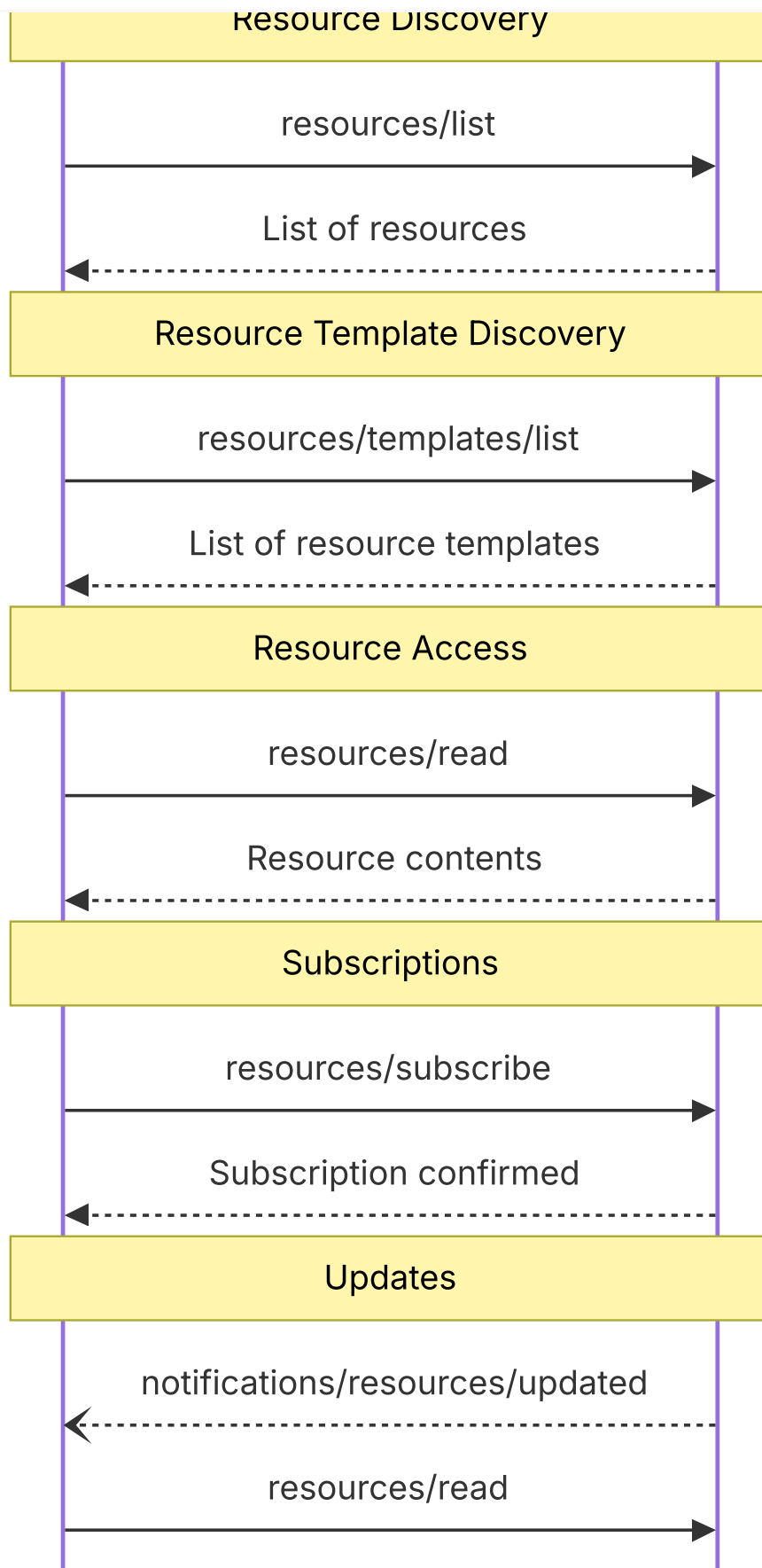


Model Context Protocol





Model Context Protocol





Model Context Protocol

5. Data Types

5.1 Resource

A resource definition includes:

- `uri` : Unique identifier for the resource
- `name` : The name of the resource.
- `title` : Optional human-readable name of the resource for display purposes.
- `description` : Optional description
- `mimeType` : Optional MIME type
- `size` : Optional size in bytes

5.2 Resource Contents

Resources can contain either text or binary data:

5.2.1 Text Content

```
{  
  "uri": "file:///example.txt",  
  "mimeType": "text/plain",  
  "text": "Resource content"  
}
```



Model Context Protocol

```
"mimeType": "image/png",  
"blob": "base64-encoded-data"  
}
```

5.3 Annotations

Resources, resource templates and content blocks support optional annotations that provide hints to clients about how to use or display the resource:

- **audience** : An array indicating the intended audience(s) for this resource. Valid values are `"user"` and `"assistant"`. For example, `["user", "assistant"]` indicates content useful for both.
- **priority** : A number from 0.0 to 1.0 indicating the importance of this resource. A value of 1 means "most important" (effectively required), while 0 means "least important" (entirely optional).
- **lastModified** : An ISO 8601 formatted timestamp indicating when the resource was last modified (e.g., `"2025-01-12T15:00:58Z"`).

Example resource with annotations:

```
{  
  "uri": "file:///project/README.md",  
  "name": "README.md",  
  "title": "Project Documentation",  
  "mimeType": "text/markdown",  
  "annotations": {  
    "audience": ["user"],  
    "priority": 0.8,  
    "lastModified": "2025-01-12T15:00:58Z"  
  }  
}
```



Model Context Protocol

- Display modification times or sort by recency

6. Common URI Schemes

The protocol defines several standard URI schemes. This list not exhaustive—implementations are always free to use additional, custom URI schemes.

6.1 https://

Used to represent a resource available on the web.

Servers **SHOULD** use this scheme only when the client is able to fetch and load the resource directly from the web on its own—that is, it doesn't need to read the resource via the MCP server.

For other use cases, servers **SHOULD** prefer to use another URI scheme, or define a custom one, even if the server will itself be downloading resource contents over the internet.

6.2 file://

Used to identify resources that behave like a filesystem. However, the resources do not need to map to an actual physical filesystem.

MCP servers **MAY** identify file:// resources with an **XDG MIME type**, like `inode/directory`, to represent non-regular files (such as directories) that don't otherwise have a standard MIME type.

6.3 git://

Git version control integration.



Model Context Protocol

7. Error Handling

Servers **SHOULD** return standard JSON-RPC errors for common failure cases:

- Resource not found: `-32002`
- Internal errors: `-32603`

Example error:

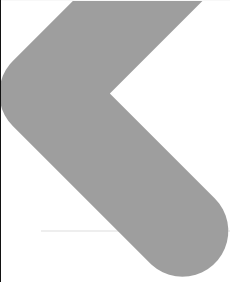
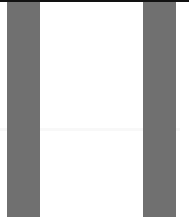
```
{
  "jsonrpc": "2.0",
  "id": 5,
  "error": {
    "code": -32002,
    "message": "Resource not found",
    "data": {
      "uri": "file:///nonexistent.txt"
    }
  }
}
```

8. Security Considerations

1. Servers **MUST** validate all resource URIs
2. Access controls **SHOULD** be implemented for sensitive resources
3. Binary data **MUST** be properly encoded
4. Resource permissions **SHOULD** be checked before operations



Model Context Protocol



Copyright © Model Context Protocol a Series of LF Projects, LLC.
For web site terms of use, trademark policy and other project policies please see
<https://lfprojects.org>.